AETHELRED

Technical Whitepaper v2.0

A Sovereign Layer 1 Blockchain for Verifiable AI Computation

Aethelred Foundation
Abu Dhabi Global Market, Al Maryah Island
Abu Dhabi, United Arab Emirates

March 2026

CONFIDENTIAL

Table of Contents

Abstract

Table of Contents

1. Introduction

- 1.1 The Problem
- 1.2 The Solution
- 1.3 Key Contributions

2. Design Philosophy

- Pillar 1: Useful Consensus
- Pillar 2: Quantum Immunity
- Pillar 3: Sovereign Privacy
- Pillar 4: AI-Native Execution
- Pillar 5: Deflationary Economics
- Pillar 6: Bi-Cameral Governance
- Pillar 7: Zero-Copy Bridging
- Pillar 8: Vector Storage

3. System Architecture

- 3.1 Layered Design
- 3.2 Technology Stack
- 3.3 Cosmos SDK Modules
- 3.4 Node Types

4. Consensus: Proof-of-Useful-Work

- 4.1 Overview
- 4.2 ABCI++ Integration
- 4.3 VRF-Based Job Assignment
- 4.4 Useful Work Scoring
- 4.5 Anti-Gaming Detection
- 4.6 Work Categories

5. Verification Framework

- 5.1 Multi-Modal Verification
- 5.2 On-Chain ZK Verification
- 5.3 Verification Orchestrator
- 5.4 TEE Attestation Binding
- 5.5 Model Registry

6. Digital Seals

6.1 Definition

6.2 Structure

6.3 Minting Conditions

6.4 Cross-Chain Relay

6.5 Oracle Interface

7. Execution Environment

7.1 Aethelred Virtual Machine

7.2 Tensor Precompiles

7.3 Cryptographic Precompiles

7.4 System Contracts

7.5 Vector Vault

8. Post-Quantum Cryptography

8.1 Threat Model

8.2 Algorithm Suite

8.3 Hybrid Signature Format

8.4 Verification Logic (Quantum-First)

8.5 Key Management

9. Sovereign Data Model

9.1 Jurisdiction-Aware Computation

9.2 Supported Jurisdictions

9.3 Data Classification

9.4 Regulatory Sandbox

10. Cross-Chain Architecture

10.1 Ethereum Bridge

10.2 Institutional Stablecoin Infrastructure

10.3 IBC Interoperability

11. Tokenomics

11.1 Token Overview

11.2 Allocation

11.3 Fixed Supply — No Inflation

11.4 Fee Market

11.5 Fee Distribution

11.6 Deflationary Mechanisms

11.7 Staking Economics

11.8 Slashing

12. Fundraising & TGE

12.1 Fundraising Rounds

12.2 Token Generation Event

12.3 Use of Proceeds

13. Community Distribution

13.1 Airdrop Program (Seals)

13.2 Tier Structure

13.3 Points Accrual

14. Market Making & Liquidity

14.1 Strategy

14.2 Exchange Strategy

15. Governance

15.1 Bi-Cameral Structure

15.2 Proposal Types

15.3 Parameter Governance Tiers

15.4 Governance Controls

15.5 Aethelred Improvement Proposals (AIPs)

15.6 Voting Power Multipliers

16. Security Architecture

16.1 Threat Model

16.2 Production Readiness

16.3 Compile-Time Security

16.4 Formal Verification

16.5 Audit Status

16.6 Fuzzing

17. Developer Ecosystem

17.1 Multi-Language SDKs

17.2 CLI Tools

17.3 Development Environment

17.4 Load Testing

18. Performance

18.1 Targets and Achievements

18.2 Scalability Roadmap

19. Regulatory Compliance

19.1 ADGM DLT Foundation

19.2 Token Compliance

19.3 Smart Contract Governance

19.4 Data Protection

20. Roadmap & Key Milestones

20.1 Timeline

20.2 Chain Maturity

21. Competitive Landscape

21.1 L1 Launch Benchmarks

21.2 Positioning

22. Risk Factors

23. Conclusion

24. References

Abu Dhabi Global Market, Al Maryah Island, Abu Dhabi, UAE

Abstract

Aethelred is a sovereign Layer 1 blockchain purpose-built for verifiable artificial intelligence computation. The protocol introduces Proof-of-Useful-Work (PoUW), a novel consensus mechanism where validators earn rewards by performing and cryptographically attesting to real AI inference workloads rather than solving arbitrary hash puzzles. Every computation produces a Digital Seal — an immutable, on-chain proof of verified intelligence that is relayable across IBC-compatible chains.

The protocol is built on Cosmos SDK and CometBFT with deep ABCI++ integration, a Rust-based execution environment with 50+ native AI tensor precompiles, hybrid post-quantum cryptography (ECDSA secp256k1 + Dilithium3/ML-DSA-65), and a multi-TEE verification framework supporting Intel SGX, AWS Nitro, AMD SEV-SNP, and NVIDIA H100 Confidential Computing. A complementary suite of Solidity smart contracts provides an EVM-compatible bridge, institutional stablecoin routing, timelocked governance, and a compliance-first token with OFAC-compatible enforcement.

Aethelred targets instant finality at sub-2-second block times with throughput of 12,500 transfers per second and 650 verified compute jobs per second, secured by a **fixed supply of 10 billion AETHEL tokens** with zero inflation, a multi-layered deflationary burn mechanism, and a four-tier slashing framework that ensures the cost of attacking the network always exceeds the potential gain.

The protocol is raising $16 million across four funding rounds (Seed, Community Echo, Strategic, Exchange) at valuations from $100M to $300M FDV, with a TGE target of $1 billion FDV at $0.10 per AETHEL during Abu Dhabi Finance Week in December 2026, followed by mainnet launch in Q1 2027.

Table of Contents

1. Introduction
2. Design Philosophy
3. System Architecture
4. Consensus: Proof-of-Useful-Work
5. Verification Framework
6. Digital Seals
7. Execution Environment
8. Post-Quantum Cryptography
9. Sovereign Data Model
10. Cross-Chain Architecture
11. Tokenomics
12. Fundraising & TGE
13. Community Distribution
14. Market Making & Liquidity
15. Governance
16. Security Architecture
17. Developer Ecosystem
18. Performance
19. Regulatory Compliance
20. Roadmap & Key Milestones
21. Competitive Landscape
22. Risk Factors

23. Conclusion
24. References

1. Introduction

1.1 The Problem

The rapid proliferation of artificial intelligence has created an asymmetry between the production and verification of AI outputs. Large language models, computer vision systems, and financial prediction engines produce results that humans and downstream systems consume without any cryptographic guarantee of correctness, provenance, or reproducibility. This verification gap has three dimensions:

Trust. When an AI model produces a credit score, a medical diagnosis, or a fraud detection alert, how does the consuming party verify that the claimed model was actually used, that the input was processed faithfully, and that the output was not tampered with? Today, the answer is organisational trust — a fundamentally unscalable and opaque mechanism.

Waste. Proof-of-Work blockchains expend enormous computational resources on arbitrary hash puzzles that produce no useful output beyond consensus. Meanwhile, the demand for AI inference is growing exponentially, creating a parallel infrastructure of GPU clusters that could be cryptographically attested instead.

Sovereignty. Existing AI infrastructure concentrates in a handful of cloud providers and jurisdictions. Enterprises, governments, and individuals in regulated markets — particularly in the Middle East, Southeast Asia, and Europe — require verifiable computation that respects data sovereignty, regulatory compliance, and jurisdictional boundaries.

1.2 The Solution

Aethelred resolves these challenges through a vertically integrated blockchain stack where:

1. **Validators perform real AI inference** as their consensus contribution, replacing hash puzzles with productive computation.
2. **Every inference is cryptographically attested** via Trusted Execution Environments (TEEs) and optionally proved via zero-knowledge machine learning (zkML) proofs.
3. **Consensus produces Digital Seals** — on-chain attestation records that serve as portable, verifiable certificates of AI computation.
4. **Data sovereignty is enforced at the protocol level** through jurisdiction-aware compute, regulatory-compliant smart contracts, and a compliance-first token design.

1.3 Key Contributions

This whitepaper presents the following technical contributions:

- **Proof-of-Useful-Work (PoUW):** A consensus mechanism that embeds AI inference verification directly into the CometBFT vote extension pipeline, achieving both consensus finality and computation attestation in a single block cycle.
- **Hybrid Post-Quantum Signatures:** A dual ECDSA + Dilithium3 signature scheme with a governance-activated Panic Mode for instant quantum emergency response.
- **Tensor Precompiles:** Over 50 native EVM precompiles for AI operations (MatMul, Attention, Softmax, Convolution) with logarithmic gas scaling, enabling on-chain model inference at orders of magnitude lower cost than general-purpose EVM execution.
- **Sovereign Data Framework:** A jurisdiction-aware data classification system that enforces data residency, encryption, and compliance at the consensus level.
- **Congestion-Squared Burn:** A novel deflationary mechanism where the token burn rate scales quadratically with network utilisation, creating exponentially increasing scarcity under high demand

— amplified by a fixed supply with zero inflation.

2. Design Philosophy

Aethelred is architected around eight foundational pillars — collectively termed **The Octagon** — each addressing a critical dimension of a verifiable AI compute network.

Pillar 1: Useful Consensus

Validators perform productive AI inference rather than wasteful hash computation. The PoUW mechanism dedicates approximately 80% of validator compute to AI workloads and 20% to consensus overhead, transforming blockchain security expenditure into a global pool of verified intelligence.

Pillar 2: Quantum Immunity

The protocol employs a hybrid cryptographic scheme combining classical ECDSA secp256k1 with post-quantum Dilithium3 (NIST FIPS 204 / ML-DSA-65) and Kyber1024 (NIST FIPS 203 / ML-KEM-1024). A governance-activated Panic Mode can instantly drop ECDSA verification if quantum computing reaches the threat threshold, preserving network security without a hard fork.

Pillar 3: Sovereign Privacy

All computation can execute within hardware Trusted Execution Environments (Intel SGX, AWS Nitro, AMD SEV-SNP, NVIDIA H100 Confidential Computing). Data enters an enclave encrypted, is processed in isolation, and exits as an attested result — the validator never sees the plaintext input. This enables "Blind Compute" where sensitive data (medical records, financial models, personal information) is processed without exposure.

Pillar 4: AI-Native Execution

Rather than forcing AI operations through generic EVM bytecode, Aethelred provides over 50 native precompiled contracts for tensor operations — matrix multiplication, transformer attention, convolution, activation functions, quantisation — with logarithmic gas scaling that makes on-chain AI inference economically viable.

Pillar 5: Deflationary Economics

AETHEL has a fixed supply of 10 billion tokens with zero inflation. A multi-layered burn mechanism (base fee burn, congestion-squared burn, adaptive quadratic burn) creates permanent, irreversible supply reduction that accelerates with network utilisation. Every burned token makes the remaining supply permanently scarcer.

Pillar 6: Bi-Cameral Governance

A two-chamber governance model — House of Tokens (stake-weighted voting) and House of Sovereigns (one-validator-one-vote) — prevents both plutocratic capture and majority tyranny, ensuring that economic stakeholders and infrastructure operators both have voice.

Pillar 7: Zero-Copy Bridging

A lock-and-mint bridge with 7-day challenge periods, 64-block Ethereum finality, guardian multi-signature oversight, per-block mint ceilings, and 27-day timelocked upgrades provides secure cross-chain asset transfer. IBC compatibility enables native interoperability with the broader Cosmos ecosystem.

Pillar 8: Vector Storage

An on-chain Vector Vault provides HNSW-indexed semantic storage supporting up to 10 million vectors per namespace with cosine similarity search, enabling AI applications to store and retrieve embeddings natively on-chain without external dependencies.

3. System Architecture

3.1 Layered Design

Aethelred is structured as a six-layer stack:

```
+-----------------------------------------------------------------+
|  APPLICATION LAYER                                               |
|  Python SDK  |  TypeScript SDK  |  Rust SDK  |  Go SDK          |
+-----------------------------------------------------------------+
|  EXECUTION LAYER                                                 |
|  Aethelred VM (EVM++)  |  AI Precompiles  |  WASM Runtime        |
|  Tensor Ops  |  Helix DSL  |  System Contracts                  |
+-----------------------------------------------------------------+
|  CONSENSUS LAYER                                                 |
|  CometBFT + ABCI++  |  PoUW Job Router  |  VRF Election          |
|  Vote Extensions  |  Digital Seal Minting  |  Slashing          |
+-----------------------------------------------------------------+
|  VERIFICATION LAYER                                              |
|  TEE Attestation  |  zkML Proofs (5 systems)  |  Hybrid Cross-   |
|  Intel SGX | AWS Nitro | AMD SEV | H100 CC    |  Validation      |
+-----------------------------------------------------------------+
|  DATA LAYER                                                      |
|  State Merkle  |  Vector Vault (HNSW)  |  Bridge Proofs          |
+-----------------------------------------------------------------+
|  HARDWARE LAYER                                                  |
|  Intel SGX DCAP | AMD SEV-SNP | AWS Nitro | NVIDIA H100 CC       |
+-----------------------------------------------------------------+
```

3.2 Technology Stack

Component	Technology	Language
Consensus	CometBFT + ABCI++	Go 1.22
Application	Cosmos SDK v0.50	Go 1.22
Virtual Machine	EVM++ with AI Precompiles	Rust 1.75
Cryptography	ECDSA + Dilithium3 + Kyber1024	Rust 1.75
Smart Contracts	ERC-20, UUPS Proxy, EIP-712	Solidity 0.8.20
Bridge	Lock-and-Mint with Guardian Multi-Sig	Solidity + Rust
SDKs	Multi-language (Python, TypeScript, Rust, Go)	Multiple
TEE Integration	SGX DCAP, Nitro NSM, SEV-SNP	Rust 1.75

3.3 Cosmos SDK Modules

The application registers four custom Cosmos SDK modules alongside the standard module set:

Module	Store Key	Purpose
`x/pouw`	`pouw`	Proof-of-Useful-Work job scheduling, validator management, tokenomics, slashing
`x/seal`	`seal`	Digital Seal lifecycle — creation, storage, verification, IBC relay
`x/verify`	`verify`	ZK proof verification, TEE attestation validation, circuit registry
`x/ibc`	`ibc`	Cross-chain Digital Seal relay via IBC protocol

Additional auxiliary modules:

- `insurance` — Escrow and appeal tribunal for disputed slashing events
- `sovereigncrisis` — Emergency halt controls for bridge and PoUW allocation

3.4 Node Types

Type	Min Stake	Hardware	Purpose
Validator	100,000 AETHEL	TEE-capable GPU server	Consensus + AI inference
Sentry	None	Standard server	DDoS shield for validators
Archive	None	10+ TB storage	Full historical state
Light	None	Minimal	Header-only verification

4. Consensus: Proof-of-Useful-Work

4.1 Overview

Proof-of-Useful-Work extends CometBFT's ABCI++ interface to embed AI inference verification directly into the consensus voting cycle. Each block contains not only traditional transactions but also cryptographically attested computation results, achieving both consensus finality and AI verification in a single sub-2-second cycle.

4.2 ABCI++ Integration

The PoUW consensus operates through four ABCI++ handler stages:

Stage 1 — ExtendVote. During the voting phase, each validator:

1. Queries the job scheduler for assigned AI inference tasks.
2. Executes each job inside a TEE and/or generates a zkML proof.
3. Constructs a `VoteExtension` containing `ComputeVerification` results — each with the job ID, model hash, input hash, output hash (SHA-256), TEE attestation quote, and optional ZK proof.

4. Signs the extension with Ed25519 (application-level defence-in-depth).
5. Returns the extension within hard caps: maximum 100 verifications, 5 MB size limit, and 10-second wall-clock budget.

Verifications are sorted deterministically by job ID before signing, ensuring two validators with identical job sets produce identical extension bytes.

Stage 2 — VerifyVoteExtension. Each validator validates incoming extensions from peers:

- Signature verification against the validator's consensus public key.
- Timestamp skew validation (maximum 10 minutes past, 1 minute future relative to block time).
- TEE attestation schema validation (platform-specific quote format, measurement registry lookup, binding hash verification).
- ZK proof structural validation (proof system, minimum size, verifying key hash).
- On-chain job existence verification (preventing phantom job votes).

In production mode, unsigned extensions, simulated TEE platforms, and attestations without block height and chain ID binding are rejected.

Stage 3 — PrepareProposal. The block proposer:

1. Retrieves vote extensions from the consensus-provided commit (not the local cache, ensuring determinism).
2. Aggregates verifications by (job ID, output hash), accumulating voting power.
3. For jobs where agreement power exceeds the 67% BFT threshold, creates `SealCreationTx` entries.
4. Injects seal creation transactions at the front of the block, capped at 50 per block and 5 MB total proof budget.
5. Optionally decrypts encrypted mempool transactions to prevent front-running.

Stage 4 — ProcessProposal. All validators verify the proposed block:

1. Record validator liveness from the last commit.
2. Validate all injected seal creation transactions against the consensus evidence.
3. Enforce per-block caps on injected transactions and total proof bytes.
4. In production mode, verify computation finality against the vote extension cache (when available) while gracefully degrading after node restarts when the cache is empty.

4.3 VRF-Based Job Assignment

Jobs are assigned to validators using a Verifiable Random Function (VRF) to ensure unpredictable, verifiable, and uniform distribution:

```
VRF_Output = VRF.Prove(Validator_SK, Epoch_Seed || Slot_Number)
```

The VRF implementation uses ECVRF-SECP256K1-SHA256-SSWU (IETF draft-irtf-cfrg-vrf-15) with RFC 9380 constant-time hash-to-curve via Simplified SWU with 3-isogeny. The domain separation tag is `"Aethelred-PoUW-VRF-v1_XMD:SHA-256_SSWU_RO_"`.

4.4 Useful Work Scoring

Each validator accumulates a Useful Work Score based on verified AI computations:

```
UW_Score = Sum(UWU_i * Category_Multiplier * Time_Decay)
```

where `UWU_i` is the Useful Work Unit for each completed job, the category multiplier reflects the societal value of the computation (1.0x for General, 1.4x for Finance, 1.6x for Research, 1.8x for Healthcare), and the time decay `0.95^(epochs_since)` ensures recent work is weighted more heavily.

The score feeds into leader election via a logarithmic multiplier:

```
UW_Multiplier = min(1 + log10(1 + UW_Score) / 6, 5.0)
Election_Threshold = (MAX_VRF * Stake * UW_Multiplier) / Total_Weighted_Stake
```

This design rewards both capital commitment (stake) and productive contribution (useful work), with logarithmic scaling preventing pure capital domination while rewarding computational productivity.

4.5 Anti-Gaming Detection

The election mechanism includes heuristic anti-gaming detection:

- Single category dominance (>95% of work in one category) is flagged.
- Single verification method dominance (>90% re-execution) is flagged.
- A Gini coefficient measures category diversity.
- Three or more flags trigger investigation and potential slashing.

4.6 Work Categories

Category	Reward Multiplier	TEE Required	Compliance Level
General	1.0x	No	Minimal
Finance	1.4x	Yes	High
Research	1.6x	No	Standard
Healthcare	1.8x	Yes	Maximum

5. Verification Framework

5.1 Multi-Modal Verification

Aethelred supports three verification modalities, selectable per job:

TEE Attestation. The computation runs inside a hardware-isolated enclave (Intel SGX, AWS Nitro, AMD SEV-SNP, or NVIDIA H100 Confidential Computing). The TEE produces a cryptographic attestation document binding the enclave measurement (code hash), input hash, output hash, block height, and chain ID. The attestation is verified against an on-chain registry of trusted measurements.

Zero-Knowledge ML Proofs (zkML). A mathematical proof is generated demonstrating that a specific model was applied to a specific input to produce a specific output, without revealing the model weights or input data. Five proof systems are supported:

System	Min Proof Size	Gas Multiplier	Primary Use Case
EZKL	256 bytes	2x	ML model inference
RISC Zero	512 bytes	4x	General zkVM computation
Plonky2	256 bytes	3x	FRI-based, fast proving
Halo2	384 bytes	2x	IPA commitment, no trusted setup
Groth16	192 bytes	1x	Most compact proofs

Hybrid Verification. Both TEE attestation and a zkML proof are generated for the same computation and cross-validated: the TEE output hash must match the zkML public inputs' output commitment. A

mismatch fails both results, providing defence-in-depth.

5.2 On-Chain ZK Verification

The `x/verify` module implements deterministic, gas-metered proof verification with nine security invariants:

- **ZK-01 (Size Gate):** Proof size enforced against global maximum (1 MB) and circuit-specific limits.
- **ZK-02 (Gas Calculation):** `gas = base_gas + (proof_size *gas_per_byte)* system_multiplier * circuit_multiplier`.
- **ZK-03 (Registry Enforcement):** Unregistered circuits rejected in production.
- **ZK-04 (Circuit Registry):** In-memory cache rebuilt atomically from on-chain state at each BeginBlock.
- **ZK-05 (VK Hash Match):** Verifying key hash compared against registered circuit.
- **ZK-06 (Domain Binding):** Anti-replay via `SHA-256(job_id || chain_id || block_height)` embedded in the last 32 bytes of public inputs.
- **ZK-08 (Groth16 Subgroup):** BN254 point-at-infinity and field modulus checks for all proof components.
- **ZK-09 (Fail-Closed):** If no registered verifier exists for a proof system and simulated mode is disabled, verification fails immediately.

An EVM precompile at address `0x0300` exposes ZK verification to smart contracts.

5.3 Verification Orchestrator

The off-chain orchestrator coordinates verification with:

- **Parallel execution:** TEE and zkML verification run concurrently for hybrid jobs.
- **LRU caching:** Successful verification results are cached (1024 entries, 5-minute TTL) keyed by `SHA-256(model_hash || input_hash || output_hash || verification_type)`.
- **Circuit breakers:** Independent breakers for prover, verifier, TEE executor, and attestation verifier prevent cascade failures.
- **Constant-time comparisons:** All hash comparisons use `crypto/subtle.ConstantTimeCompare` to prevent timing side-channels.

5.4 TEE Attestation Binding

In production mode, TEE attestations are bound to the specific block and chain:

```
UserData = SHA-256(output_hash || LE64(block_height) || chain_id)
```

This prevents cross-chain and cross-block replay of attestation documents. Attestation timestamps are validated within a 10-minute past / 1-minute future window relative to block time.

5.5 Model Registry

An on-chain model registry manages the lifecycle of AI models available for verified inference:

```
Model Status: pending -&gt; compiling -&gt; active -&gt; inactive
                                \-&gt; failed
```

Models registered with ONNX format are automatically compiled to zkML circuits (EZKL) when auto-compilation is enabled. The registry stores model hashes, circuit hashes, verifying keys, TEE measurements, and per-circuit gas multipliers.

6. Digital Seals

6.1 Definition

A Digital Seal is an immutable, on-chain attestation record minted when a supermajority (>= 2/3 + 1) of validators agree on a computation result. Seals serve as portable, verifiable certificates of AI inference.

6.2 Structure

Each Digital Seal contains:

- **Seal ID:** Globally unique identifier `{chain_id}/{block_height}/{job_id}`.
- **Computation Binding:** Model hash, input hash, output hash (all SHA-256).
- **Consensus Evidence:** Validator attestations (TEE quotes and/or ZK proofs), agreement power, total power.
- **Provenance:** Block height, timestamp, chain ID.
- **BLS Aggregate Signature:** A single 96-byte BLS12-381 signature replacing N individual validator signatures.

6.3 Minting Conditions

A seal is minted if and only if:

1. The job exists on-chain and is in a valid state.
2. Agreement power >= `ceil(total_power * 67 / 100)`.
3. All agreeing validators produced identical output hashes.
4. No existing seal exists for the same job ID.

6.4 Cross-Chain Relay

Digital Seals are relayable via:

- **EVM Precompile** at address `0x0400` for on-chain verification by smart contracts.
- **IBC Protocol** on port `aethelred.seal` for cross-chain relay to any IBC-compatible blockchain.

6.5 Oracle Interface

The `IAethelredOracle` interface enables smart contracts to query:

- `getResult(jobId)` — ABI-encoded computation result.
- `verifySeal(sealId)` — Boolean seal validity check.
- `verifySealWithOutput(sealId, outputCommitment)` — Seal validity plus output commitment match.
- `getAttestations(sealId)` — Full TEE attestation records.
- `getModelInfo(modelId)` — Model commitment and version.

7. Execution Environment

7.1 Aethelred Virtual Machine

The Aethelred VM extends EVM compatibility with native AI operation support. The execution environment is implemented in Rust for memory safety and performance, with three execution backends:

1. **EVM++ Interpreter** — Full EVM compatibility with additional precompiled contracts.
2. **WASM Runtime** — Sandboxed WebAssembly execution for general-purpose computation.

3. **Helix DSL** — A domain-specific language for sovereign AI functions with built-in compliance annotations.

7.2 Tensor Precompiles

Fifty-plus native precompiled contracts at addresses `0x1000–0x10FF` provide hardware-accelerated AI operations:

Address Range	Category	Key Operations
`0x1000–0x100F`	Matrix	MatMul, BatchMatMul, OuterProduct
`0x1010–0x101F`	Element-wise	Add, Mul, Div, Pow, Exp, Log
`0x1020–0x102F`	Activations	ReLU, GELU, SiLU, Sigmoid, Softmax, Mish
`0x1030–0x103F`	Reductions	Sum, Mean, Max, ArgMax, Norm
`0x1040–0x104F`	Convolution	Conv2D, MaxPool2D, GlobalAvgPool
`0x1050–0x105F`	Transformer	Attention, MultiHeadAttention, FlashAttention, LayerNorm, RMSNorm, RotaryEmbedding
`0x1060–0x106F`	Quantisation	Quantise, Dequantise, QuantisedMatMul
`0x1070–0x107F`	Manipulation	Reshape, Transpose, Concat, Gather, Scatter
`0x1080–0x108F`	Special	Embedding, CrossEntropy, TopK, NMS
`0x10F0–0x10FF`	Verification	VerifyModelHash, VerifyOutputHash, GenerateProof

Gas scaling follows a logarithmic model — `gas = base + log2(tensor_size) * 10` — preventing gas bombs while making large-scale inference economically viable.

Eight data types are supported: Float32, Float16, BFloat16, Int8, UInt8, Int32, Int64, Bool. Operations can target CPU, GPU, or TEE device backends.

7.3 Cryptographic Precompiles

Native VM precompiles provide hardware-accelerated cryptographic operations:

Precompile	Purpose
SHA-256	Standard hashing
ECDSA Recovery	Ethereum-compatible address recovery
Dilithium3 Verify	Post-quantum signature verification

Precompile	Purpose
Dilithium5 Verify	High-security PQ signature verification
Kyber1024 Decapsulate	Post-quantum key exchange
Kyber1024 Decapsulate	High-security PQ key exchange
Hybrid Verify	Combined ECDSA + Dilithium verification with threat-level awareness

7.4 System Contracts

Two system contracts manage core economic operations within the VM:

Job Registry. Manages the full AI inference job lifecycle:

```
SUBMITTED -&gt; ASSIGNED -&gt; PROVING -&gt; VERIFYING -&gt; VERIFIED -&gt; SETTLED
```

Key parameters: minimum bid 0.1 AETHEL, maximum 100,000 AETHEL, default SLA timeout 1 hour (adjustable by priority), 5% cancellation fee, 10% late penalty, maximum 100,000 concurrent active jobs.

Tokenomics Contract. Enforces distribution schedules and the adaptive burn engine on the VM layer, with u128 arithmetic for overflow safety.

7.5 Vector Vault

An on-chain vector storage system provides HNSW-indexed (Hierarchical Navigable Small World) semantic search:

- **Capacity:** Up to 10,000,000 vectors per namespace.
- **Index Types:** Flat (exact), IVF (approximate), HNSW (default, m=16, ef_construction=200), PQ (100x compression), SQ (4-8x compression).
- **Embedding Models:** Support for OpenAI Ada-002, OpenAI 3-Small/Large, Cohere v3, Sentence Transformers, CLIP, BioMedLM, FinBERT, and custom models.
- **Search:** Cosine similarity with epsilon-safe denominator, filterable by tags, content type, date range, and owner.
- **Access Control:** Per-namespace owner, reader list, public/private flag, optional encryption.
- **Document Processing:** Automatic chunking (fixed, semantic, sentence, recursive) with configurable chunk size (default 512 tokens) and overlap (default 50 tokens).

8. Post-Quantum Cryptography

8.1 Threat Model

Aethelred's cryptographic architecture addresses three threat scenarios:

1. **Pre-Quantum (current):** Both ECDSA and Dilithium are verified for every signature, providing dual security.
2. **Quantum Threat (NIST Level 3-4):** ECDSA verification becomes optional; Dilithium remains mandatory.
3. **Post-Quantum (Q-Day):** Governance activates Panic Mode — ECDSA verification is dropped entirely, and only Dilithium is verified.

8.2 Algorithm Suite

Algorithm	Standard	Security Level	Purpose
ECDSA secp256k1	NIST SP 800-186	128-bit classical	Ethereum ecosystem compatibility
Dilithium3 (ML-DSA-65)	NIST FIPS 204	192-bit classical, 128-bit quantum	Primary signature scheme
Dilithium5 (ML-DSA-87)	NIST FIPS 204	256-bit classical, 192-bit quantum	High-security option
Kyber1024 (ML-KEM-1024)	NIST FIPS 203	256-bit classical, 192-bit quantum	High-security key exchange
Kyber768 (ML-KEM-768)	NIST FIPS 203	192-bit classical, 128-bit quantum	Key encapsulation

8.3 Hybrid Signature Format

```
[Version: 1B] [Marker 0x03: 1B] [ECDSA r||s: 64B] [Separator 0xFF: 1B]
[Dilithium Level: 1B] [Dilithium Sig Length: 2B] [Dilithium Sig: ~3.3KB]
[Optional Metadata: timestamp + chain_id]
```

Total hybrid signature size: approximately 3.4 KB (Dilithium3) versus 64 bytes for ECDSA alone. This overhead is justified by quantum resistance.

8.4 Verification Logic (Quantum-First)

Verification always proceeds Dilithium-first:

1. Validate format and security level.
2. Check timestamp freshness and chain ID binding.
3. **Always** verify Dilithium signature (primary security layer).
4. Verify ECDSA signature **unless** Panic Mode is active.

This "quantum-first" approach ensures that even if a future quantum computer breaks ECDSA in real-time, the Dilithium verification has already succeeded.

8.5 Key Management

All secret key types implement `ZeroizeOnDrop` — key material is securely erased from memory when no longer needed. Debug output is redacted. HSM integration (AWS CloudHSM, Thales Luna, YubiHSM 2) is supported for mainnet validators, with SoftHSM available for development. The HSM preflight check validates connectivity, session health, test signing, key label resolution, and failover readiness.

9. Sovereign Data Model

9.1 Jurisdiction-Aware Computation

Aethelred wraps all data in `Sovereign<T>` containers that bind encrypted payloads to jurisdictions, data classifications, compliance requirements, and access policies. Computation respects these bindings at the consensus level — a job tagged with UAE data sovereignty will only be scheduled on validators operating within ADGM-compliant infrastructure.

9.2 Supported Jurisdictions

Jurisdiction	Framework	Key Requirements
UAE (ADGM/DIFC)	Federal Decree-Law 45/2021	Data residency, CBUAE compliance
Saudi Arabia	PDPL	Data sovereignty
European Union	GDPR	Articles 17, 25, 44 compliance
United Kingdom	UK-GDPR	Post-Brexit data protection
United States	CCPA, HIPAA, SOX	Sector-specific compliance
Singapore	PDPA, MAS TRM	Section 26 transfer limitation
China	PIPL	Cross-border data restrictions
Global	Minimal	Baseline compliance

9.3 Data Classification

Level	Encryption	TEE Required	Audit Retention
Public	Optional	No	30 days
Internal	At-rest	No	90 days
Confidential	E2E	Recommended	7 years
Sensitive	E2E + PQ	Required	7 years, immutable
Restricted	E2E + PQ + MPC	Attested	Legal hold
Top Secret	Multi-party	Air-gapped	Indefinite

9.4 Regulatory Sandbox

The protocol includes a regulatory sandbox module that implements compliance checking against actual legal citations (GDPR Articles, UAE Federal Decree-Law provisions, PDPA Sections, HIPAA CFR references). The sandbox supports:

- OFAC SDN, EU, and UN sanctions screening.
- Consent management with purpose-specific tracking and withdrawal support.
- Data transfer audit trails with source/destination location tracking.
- Violation detection for cross-border transfer, unencrypted PII, missing consent, retention violations, and sanctions violations.

10. Cross-Chain Architecture

10.1 Ethereum Bridge

The `AethelredBridge` smart contract implements a lock-and-mint bridge with multi-signature relayer consensus:

Deposit Flow (Ethereum to Aethelred):

1. User locks ETH or ERC-20 tokens via `depositETH()` / `depositERC20()`.
2. Relayers observe the deposit after 64 Ethereum confirmations (~12.8 minutes post-merge finality).
3. Corresponding AETHEL tokens are minted on the Aethelred chain.

Withdrawal Flow (Aethelred to Ethereum):

1. User burns AETHEL tokens on the Aethelred chain.
2. Relayers propose a withdrawal on Ethereum with vote-based consensus.
3. A 7-day challenge period allows guardians to flag fraud.
4. After the challenge period, the withdrawal is finalised.

Security Controls:

- Rate limiting: 1,000 ETH per hour for both deposits and withdrawals.
- Per-transaction bounds: 0.01 ETH minimum, 100 ETH maximum.
- Per-block mint ceiling: 10 ETH.
- Guardian multi-signature: 2-of-N for emergency operations.
- Emergency withdrawal cap: 50 ETH with 48-hour timelock.
- Upgrade timelock: Minimum 27 days, UPGRADER_ROLE self-administered.

10.2 Institutional Stablecoin Infrastructure

A zero-liquidity-pool stablecoin routing infrastructure handles institutional flows through issuer-authorised channels:

- **CCTP V2** — Circle burn-and-mint for USDC cross-chain transfers.
- **TEE Issuer Mint** — Issuer-gated attestation flow for non-CCTP stablecoins.
- **Five governance keys** (3-of-5 threshold) — Issuer, Issuer Recovery, Foundation, Auditor, Guardian.
- **Relayer bonding** — 500,000 AETHEL bond requirement for participation.
- **Circuit breakers** — Automatic pause on reserve anomaly, oracle staleness, velocity breach, or transaction limit breach.
- **Chainlink PoR integration** — Automated Proof-of-Reserve monitoring via `checkUpkeep()` / `performUpkeep()`.

10.3 IBC Interoperability

Digital Seals are relayable across IBC-compatible chains via port `aethelred.seal`, enabling verified AI computation results to be consumed by any chain in the Cosmos ecosystem.

11. Tokenomics

11.1 Token Overview

Property	Value
Name	Aethelred
Ticker	AETHEL
Total Supply	10,000,000,000 (hard cap, fixed)

Property	Value
Supply Model	Fixed — No inflation
Token Standard	ERC-20 (Upgradeable, Votes, Permit, Burnable, Pausable)
Cosmos Denomination	uaethel (6 decimals)
EVM Denomination	wei (18 decimals)
Cross-Layer Scale	1 uaethel = 10^12 wei

11.2 Allocation

Category	Share	Tokens	TGE Unlock	Cliff	Total Vest
PoUW Rewards	30%	3,000,000,000	0%	—	120 months
Core Contributors	20%	2,000,000,000	0%	6 months	48 months
Ecosystem & Grants	15%	1,500,000,000	2%	6 months	54 months
Treasury & Market Makers	6%	600,000,000	25%	—	36 months
Public Sales	7.5%	750,000,000	20%	—	18 months
Airdrop (Seals)	7%	700,000,000	25%	—	12 months
Strategic / Seed	5.5%	550,000,000	0%	12 months	36 months
Insurance Fund	5%	500,000,000	10%	—	30 months
Contingency Reserve	4%	400,000,000	0%	12 months	TBD

TGE circulating supply: **630,000,000 AETHEL (6.3%)**.

11.3 Fixed Supply — No Inflation

All 10 billion AETHEL are minted at genesis. There is no inflationary minting. The PoUW Rewards pool (3 billion AETHEL) is distributed linearly at **25,000,000 AETHEL per month** over 10 years to validators performing useful AI computation — providing economic security incentives without diluting existing holders.

11.4 Fee Market

EIP-1559-inspired dynamic pricing with AI-specific extensions:

```
Total Fee = Base Fee * Dynamic Multiplier * Priority Tier
            * Hardware Multiplier * Jurisdiction Multiplier
```

- Base fee: 0.001 AETHEL, adjusting +/-12.5% per block.
- Dynamic multiplier: 1x–5x based on congestion (70% threshold).
- Priority tiers: Standard (1x), Fast (2x), Urgent (5x).
- Hardware multipliers: Generic (1x) to H100 Confidential (2x).
- Jurisdiction multipliers: Global (1x) to Saudi Arabia (1.5x).

11.5 Fee Distribution

Bucket	Share	Purpose
Validator Rewards	40%	Reputation-scaled validator compensation

Bucket	Share	Purpose
Treasury	30%	Protocol development and grants
Burn	20%	Permanent token destruction
Insurance Fund	10%	Economic stability reserve

11.6 Deflationary Mechanisms

With a fixed supply and zero inflation, every burn permanently reduces the total supply:

1. **Base Fee Burn:** 20% of all fees permanently burned.
2. **Congestion-Squared Burn:** `Burn_Rate = Base_Fee * (1 + Congestion^2)` where `Congestion = clamp((utilisation - 0.5) / 0.5, 0, 1)`.
3. **Adaptive Quadratic Burn:** `Burn_Rate = B_min + (B_max - B_min) * (usage / capacity)^2`, bounded between 5% and 20%.

AETHEL is structurally deflationary from genesis. The total supply can only decrease, never increase.

11.7 Staking Economics

Parameter	Value
Minimum Validator Stake	100,000 AETHEL
Maximum Validators	100
Unbonding Period	21 days
Commission Range	5%–20%
Staking Target	55%
Concentration Cap	33% per validator

Reputation-scaled rewards: `scaled_reward = base_reward * (50 + reputation / 2) / 100`.

11.8 Slashing

Tier	Offense	Slash	Ban
Minor	Brief downtime	0.5%	1 day
Major	Extended downtime	10%	7 days
Fraud	False computation	50%	1 year (permanent ban)
Critical Byzantine	Coordinated attack	100%	Permanent

For a comprehensive treatment, see the companion Tokenomics document.

12. Fundraising & TGE

12.1 Fundraising Rounds

Round	Raise	Tokens	Price / AETHEL	% Supply	FDV	Timing
Seed	$1,000,000	100,000,000	$0.010	1.00%	$100M	Mar–Apr 2026
Community (Echo)	$6,000,000	400,000,000	$0.015	4.00%	$150M	Jun–Jul 2026
Strategic	$3,750,000	150,000,000	$0.025	1.50%	$250M	Sep–Oct 2026
Exchange (Binance)	$5,250,000	175,000,000	$0.030	1.75%	$300M	Nov 2026
Total	**$16,000,000**	**825,000,000**	**$0.0194 avg**	**8.25%**	—	—

12.2 Token Generation Event

Metric	Value
TGE Date	December 7–10, 2026 (Abu Dhabi Finance Week)
TGE Price Target	$0.10 / AETHEL
TGE Market Cap	$63,000,000
Fully Diluted Valuation	$1,000,000,000
TGE Circulating Supply	630,000,000 AETHEL (6.3%)
Average Investor Entry	$0.0194 / AETHEL
Investor Upside at TGE	5.15x
Treasury Runway	18+ months

12.3 Use of Proceeds

Category	Amount	Share
Engineering	$6,400,000	40%
Infrastructure	$2,400,000	15%
Security & Audits	$2,400,000	15%
Business Development	$1,600,000	10%
Legal & Compliance	$1,600,000	10%
Operations	$1,600,000	10%

13. Community Distribution

13.1 Airdrop Program (Seals)

The Seals airdrop program distributes **700,000,000 AETHEL** (7% of total supply) to community members based on a points-based engagement system.

13.2 Tier Structure

Tier	Points Required	AETHEL per User	Est. Users	Total AETHEL
Diamond	10,000+	50,000	500	25,000,000
Gold	5,000–9,999	20,000	2,000	40,000,000
Silver	1,000–4,999	5,000	10,000	50,000,000
Bronze	100–999	500	50,000	25,000,000
Subtotal	—	—	**62,500**	**140,000,000**

Additional reserves: 210M for ongoing incentives, testnet bonuses, and Echo early registration. All airdrop tokens vest with 25% TGE unlock and 75% over 12 months.

13.3 Points Accrual

Points accumulate from March 2026 through the November 2026 snapshot via:

- Social engagement (Discord, X, Telegram)
- Testnet validation and bug bounty submissions
- Content creation and community moderation
- Developer activity (dApp deployment, SDK contributions)

14. Market Making & Liquidity

14.1 Strategy

Partner	Token Loan	TGE Available	Type	Terms
Wintermute	125,000,000	125,000,000	Primary MM	3-month renewable loan + fee
GSR	75,000,000	75,000,000	Secondary MM	3-month renewable loan + fee
Exchange Listings	175,000,000	43,750,000	Listing Liquidity	Per agreement
Total	**375,000,000**	**243,750,000**	—	**~$37.5M at FDV**

Market maker tokens are **loaned, not sold** — they are returnable per agreement, ensuring they support price discovery and liquidity rather than creating sell pressure. Dual MM coverage (Wintermute + GSR) provides redundancy across exchanges.

14.2 Exchange Strategy

- **Primary:** Binance listing at TGE (November 2026 exchange round).
- **Contingency:** Bybit or OKX if Binance timeline shifts.
- **DEX:** Uniswap V3 or equivalent for DeFi accessibility.

15. Governance

15.1 Bi-Cameral Structure

House of Tokens. Voting power proportional to staked AETHEL. Open to all stakers.

House of Sovereigns. One vote per validator, independent of stake size. Provides infrastructure-operator representation that prevents pure plutocratic control.

Both chambers must approve proposals for them to pass.

15.2 Proposal Types

Type	Quorum	Pass Threshold	Chamber
Parameter Change	20%	50%	Both
Treasury Disbursement	30%	66%	Both
Validator Set	66%	66%	Sovereigns only
Protocol Upgrade	40%	75%	Both
Emergency Action	80%	80%	Sovereigns only

15.3 Parameter Governance Tiers

Tier	Override Quorum	Examples
Mutable	67%	Job timeout, base fee, max jobs per block
Locked	80%	Fee distribution, slashing penalties
Critical	90%	TEE attestation requirement
Immutable	Cannot be changed	AllowSimulated = false (one-way gate)

15.4 Governance Controls

Category	Structure	Signers	Timelock
Insurance Fund	3-of-5 Multi-sig	2 Team + 2 Advisors + 1 Auditor	7 days (>10M AETHEL)
Contingency Reserve	Team + Advisor Majority	Core team + board	30 days
Treasury	Team-controlled	CEO + CFO + CTO	14 days (>$500K)
Ecosystem Fund	4-of-7 Multi-sig	Foundation council	48 hours
Smart Contract Upgrades	Governance	UPGRADER_ROLE	27 days minimum

Category	Structure	Signers	Timelock
Key Rotation	Dual-signature	Issuer + Foundation	7 days

15.5 Aethelred Improvement Proposals (AIPs)

Formal protocol changes follow the AIP process:

Type	Scope
Core	Consensus, protocol, hard forks
Standard	Interfaces, APIs, application-level
Informational	Guidelines, best practices
Meta	AIP process itself

Lifecycle: Idea -> Draft -> Review -> Final (with Withdrawn/Replaced branches).

15.6 Voting Power Multipliers

Stakers who lock their AETHEL tokens for longer durations receive enhanced governance voting power:

Lock Duration	Voting Power Multiplier
90 days	1.0x
180 days	1.25x
365 days	1.6x
730 days	2.0x

This mechanism rewards long-term commitment to the protocol's governance, ensuring that participants with the greatest temporal alignment exercise proportionally greater influence over protocol decisions.

16. Security Architecture

16.1 Threat Model

The protocol's security model has been validated against five adversary classes:

1. **Rational Economic Attacker** — Deterred by the slashing framework where deterrence ratio > 1 for all attack vectors.
2. **State-Level Adversary** — Mitigated by geographic distribution requirements, bi-cameral governance, and the 33% validator concentration cap.
3. **MEV Extractor** — Mitigated by VRF-based job assignment, encrypted mempool bridge, and priority-based scheduling.
4. **Bridge Attacker** — Mitigated by 7-day challenge periods, 64-block Ethereum finality, rate limits, and guardian oversight.
5. **Cryptographic Adversary** — Mitigated by hybrid post-quantum cryptography with instant Panic Mode.

16.2 Production Readiness

Mainnet launch requires passing 12 production-readiness gates:

Gate	Check
G1	External audit sign-off (Trail of Bits + OtterSec)
G2	CI branch protection (6 required gates)
G3/G4	Dependency integrity (Go/Rust)
G5	Static analysis (gosec, trivy, slither, cargo-audit)
G6	Formal verification and fuzzing
G7	Fail-closed production configuration
G8	HSM preflight validation
G9	Production-like E2E topology
G10	Exploit simulation determinism
G11	Operations readiness documentation
G12	Documentation hygiene

All 12 gates pass as of March 2026.

16.3 Compile-Time Security

A three-layer defence prevents simulated verification from reaching production:

1. **Build tag** (`-tags production`): Compile-time `IsProductionBuild()` flag overrides all runtime configuration.
2. **One-way governance gate**: `AllowSimulated` can only transition from `true` to `false`; once `false`, it cannot be re-enabled by any governance action.
3. **Runtime readiness check**: The first BeginBlocker after genesis runs `RunProductionReadinessChecks()`, which panics in production mode if any check fails.

16.4 Formal Verification

Contract	Tool	Invariants
AethelredToken	Certora/Halmos	Supply cap, blacklist correctness, transfer safety
AethelredVesting	Certora/Halmos	Monotonicity, solvency, release bounds
AethelredBridge	Certora/Halmos	No double withdrawal, rate limits
Tokenomics (Rust)	Kani	Vested amount bounded
Consensus	TLA+	Safety (no conflicting blocks), liveness

16.5 Audit Status

Auditor	Scope	Status
Trail of Bits	Smart contracts, consensus, cryptography	Engaged — 2 audit cycles
OtterSec	Smart contracts, bridge, tokenomics	Engaged — 2 audit cycles
Internal Security Review	Full protocol (Go/Solidity/Rust)	Complete — 27 findings, all remediated
External VRF Consultant	VRF + protocol review	Complete — RS-01 (Critical) fixed

Internal audit results: 3 Critical, 5 High, 8 Medium, 7 Low, 6 Informational — **all 27 findings remediated and closed.**

16.6 Fuzzing

Comprehensive fuzz testing with `cargo-fuzz`, `go-fuzz`, and Foundry across three priority tiers. Coverage targets: VRF >80%, bridge >70%, token >85%, vesting >85%, vote extension parsing >90%.

17. Developer Ecosystem

17.1 Multi-Language SDKs

Python SDK. Production-grade runtime with:

- Hardware abstraction layer supporting 12 device types including 4 TEE platforms.
- Size-class memory allocator with 256 MB pool and copy-on-write semantics.
- CUDA-like async execution streams with event synchronisation.
- JIT compilation with persistent disk cache and optimisation levels O0-Ofast.
- PyTorch-compatible neural network API (Module, Linear, Embedding, MultiheadAttention, TransformerEncoderLayer).
- Full quantisation toolkit: PTQ, QAT, INT8/INT4/NF4, per-tensor/per-channel/per-group granularity.
- Chrome Trace profiling output.

TypeScript SDK. Browser and Node.js compatible with:

- PyTorch-compatible Module API with async device transfer (WebGPU/WebGL backends).
- State dict serialisation.
- Blockchain client for job submission, seal verification, governance participation.

Rust and Go SDKs. Native-performance SDKs with direct protobuf integration.

17.2 CLI Tools

`aethelredd` — The node binary (Go/Cosmos SDK). Supports full, validator, and light node modes with HSM integration.

`aeth` (Rust) — Comprehensive developer CLI with 15 command groups: interactive REPL, project scaffolding, model management (register/deploy/quantise/benchmark), job submission, seal management, validator operations, account management, network inspection, development tools (hot reload, codegen, linting, profiling), benchmarking, configuration, compliance checking, and hardware

detection.

`aeth` (TypeScript) — Complementary developer CLI focused on local development: Docker Compose orchestration, network diagnostics, offline seal verification, and wallet operations.

17.3 Development Environment

Docker Compose devnet with 10 services: 3 validators (heterogeneous TEE platforms), zkML prover with GPU passthrough, PostgreSQL 16, Redis 7, API gateway with WebSocket, block explorer, token faucet, Prometheus + Grafana monitoring.

VSCode Extension ("Aethelred Sovereign Copilot") with real-time compliance linting, cost estimation hovers, multi-regulation support (GDPR, HIPAA, UAE-DPL, CCPA, PIPL), Helix DSL syntax highlighting, and in-editor TEE simulation.

17.4 Load Testing

Deterministic, seed-based network simulation with 7 predefined scenarios:

- Simple/asymmetric/three-way network partitions.
- Single-node and multi-node eclipse attacks.
- Delayed message delivery.
- Combined partition + eclipse attacks.

BFT-aware grading system with letter grades (A+ through D) for attack resilience and recovery.

18. Performance

18.1 Targets and Achievements

Metric	Target	Achieved
Block Time	3 seconds	< 2 seconds
Finality	Instant	< 2 seconds (no reorgs)
TPS (Transfers)	10,000	12,500
TPS (Compute Jobs)	500	650
Inference Latency	< 100ms	80–150ms p50 (Llama-3 8B, batch 1)
TEE Attestation	< 5s	2.3s
ZK Proof (EZKL)	< 30s	18s (small models)

18.2 Scalability Roadmap

Phase 1 (2026–2027): Vertical scaling with 100 validators, targeting 1,000 compute jobs per second.

Phase 2 (2027–2028): Jurisdiction-specific shards via IBC, targeting 10,000 compute jobs per second. Each shard specialises in a jurisdiction (UAE, EU, US) with its own validator set while sharing a common settlement layer.

Phase 3 (2028+): Dedicated data availability layer for model weights, enabling petabyte-scale model storage. Horizontal scaling of tensor precompile execution.

19. Regulatory Compliance

19.1 ADGM DLT Foundation

Aethelred is structured as a foundation registered under the Abu Dhabi Global Market (ADGM) Distributed Ledger Technology (DLT) Foundations regime. The ADGM DLT Foundation provides a regulated legal wrapper for decentralised protocol governance, enabling:

- Legal personality for the protocol governance body.
- Compliance with UAE financial regulatory frameworks.
- Structured relationships with validators, developers, and token holders.

19.2 Token Compliance

The AETHEL token implements comprehensive compliance controls:

- **OFAC/Sanctions screening** via on-chain blacklisting (batch operations up to 200 addresses).
- **Compliance burn** with mandatory reason codes and full audit trail.
- **Transfer restrictions** for pre-TGE lock with whitelist exemptions.
- **Multi-signature requirements** for all administrative operations on production chains.

19.3 Smart Contract Governance

- **27-day minimum upgrade timelock** for bridge contracts.
- **7-day minimum delay** for governance key rotation with dual-signature requirement (issuer + foundation).
- **4-of-7 multi-signature** with 48-hour timelock on ecosystem fund disbursements.
- **3-of-5 institutional governance** for stablecoin infrastructure.
- **3-of-5 multi-sig** for Insurance Fund (2 team + 2 advisors + 1 auditor).

19.4 Data Protection

The Sovereign Data Model (Section 9) ensures compliance with data protection regulations across all supported jurisdictions, with enforcement at the consensus level rather than relying on application-layer compliance alone.

20. Roadmap & Key Milestones

20.1 Timeline

Date	Milestone	Type	Details
Mar 2026	Community Launch	Community	Discord, X, Telegram activation; Seals points program begins
Mar–Apr 2026	Seed Round	Fundraising	$1M at $100M FDV
May 1, 2026	Testnet Launch	Technical	Public testnet; validator onboarding; bug bounty live

Date	Milestone	Type	Details
Jun–Jul 2026	Community (Echo) Round	Fundraising	$6M at $150M FDV via Coinbase Echo
Sep–Oct 2026	Strategic Round	Fundraising	$3.75M at $250M FDV
Nov 2026	Exchange Round	Fundraising	$5.25M at $300M FDV (Binance primary; Bybit/OKX contingency)
Nov 2026	Airdrop Snapshot	Tokenomics	Final Seals points tally
Dec 7–10, 2026	Token Generation Event	TGE	Abu Dhabi Finance Week; airdrop distribution; exchange listings
Q1 2027	Mainnet Launch	Technical	Full network launch; dApp ecosystem live
2027	Phase 2	Scaling	Jurisdiction-specific shards via IBC; 10,000 jobs/sec target
2028	Phase 3	Scaling	Dedicated DA layer; petabyte-scale model storage

20.2 Chain Maturity

Phase	Duration	Characteristics
Launch	0–2 weeks	Bootstrap governance (33% quorum), limited proposal capacity
Early Operations	2–6 weeks	Transitional monitoring, maturity scoring
Stable	6–12 weeks	Full governance activation (40% quorum)
Mature	12+ weeks	Full decentralisation, parameter self-governance

21. Competitive Landscape

21.1 L1 Launch Benchmarks

Project	Team %	TGE Float	Airdrop %	FDV at TGE	Outcome
Sui	20%	~5%	6%	$2B	Successful but community sought larger airdrop

Project	Team %	TGE Float	Airdrop %	FDV at TGE	Outcome
Aptos	20%	~13%	51% (incl. testnet)	$1B	Significant sell pressure due to high float
Celestia	20%	~20%	7.4%	$300M	Well-received; careful community targeting
Arbitrum	~27%	~12%	12.75%	$1.4B	Successful; 2+ years of usage before TGE
Monad	TBD	TBD	TBD	TBD	Highly anticipated; similar structure expected
Aethelred	**20%**	**6.3%**	**7%**	**$1B (target)**	**Balanced; lessons learned from above**

21.2 Positioning

Aethelred does not compete with general-purpose smart contract platforms. It creates an entirely new category: a **Verifiable Compute Cloud** that sells **Verified Intelligence**. The key differentiators are:

Dimension	Aethelred	General-Purpose L1s
Consensus purpose	AI inference verification	Transaction ordering
Validator reward source	Useful AI computation	Block rewards only
Supply model	Fixed (0% inflation)	Inflationary (2–8%)
Cryptography	Hybrid PQC (ECDSA + Dilithium3)	ECDSA only
Compliance	Consensus-level enforcement	Application-layer only
AI capabilities	50+ native tensor precompiles	None / generic EVM
Data sovereignty	Jurisdiction-aware at consensus	Not supported

22. Risk Factors

Risk	Likelihood	Impact	Mitigation
TGE price dump	Medium	High	6.3% float is defensible; $37.5M MM ammunition; 80% Public Sale locked
Geopolitical (UAE conflict)	Medium	Medium	Virtual TGE backup plan; alternative jurisdictions identified
Team hiring difficulty	Low	High	20% allocation matches market standard; 6-month cliff for early liquidity
Exchange listing failure	Low	High	Binance primary + Bybit/OKX contingency; DEX backup

Risk	Likelihood	Impact	Mitigation
Testnet delay	Medium	Medium	Buffer built into timeline; contractor support available
Smart contract exploit	Low	Catastrophic	2 audit cycles (Trail of Bits + OtterSec); Insurance Fund; formal verification
Regulatory uncertainty	Medium	Medium	ADGM engagement from day one; legal counsel on retainer; compliance-first design

23. Conclusion

Aethelred represents a fundamental rethinking of blockchain consensus, transforming the security budget from wasteful computation into a global pool of verified intelligence. By embedding AI inference verification directly into the CometBFT vote extension pipeline, the protocol achieves both consensus finality and computation attestation in a single block cycle — creating Digital Seals that serve as portable, cryptographic certificates of AI integrity.

The protocol's hybrid post-quantum cryptography ensures long-term security against both classical and quantum adversaries. Its sovereign data model enforces jurisdictional compliance at the consensus level. Its fixed-supply, structurally deflationary economics create a token that becomes scarcer as the network becomes more useful. And its bi-cameral governance prevents capture by either capital or infrastructure operators.

With $16 million in planned funding, a TGE at Abu Dhabi Finance Week in December 2026, and mainnet launch in Q1 2027, Aethelred is positioned to capture the emerging market for verified AI computation — delivering trust, sovereignty, and cryptographic proof to every AI inference.

24. References

1. NIST FIPS 204 (ML-DSA / Dilithium). Module-Lattice-Based Digital Signature Standard. National Institute of Standards and Technology, 2024.

2. NIST FIPS 203 (ML-KEM / Kyber). Module-Lattice-Based Key-Encapsulation Mechanism Standard. National Institute of Standards and Technology, 2024.

3. IETF draft-irtf-cfrg-vrf-15. Verifiable Random Functions (VRFs). Internet Research Task Force, 2023.

4. RFC 9380. Hashing to Elliptic Curves. Internet Engineering Task Force, 2023.

5. EIP-1559. Fee market change for ETH 1.0 chain. Ethereum Improvement Proposals, 2021.

6. Buterin, V. et al. CometBFT: Byzantine Fault Tolerant Middleware. Interchain Foundation, 2023.

7. EZKL. Zero-Knowledge Proofs for Machine Learning Inference. https://ezkl.xyz.

8. RISC Zero. General Purpose Zero-Knowledge Virtual Machine. https://risczero.com.

9. Abu Dhabi Global Market. DLT Foundations Regulations. ADGM, 2023.

10. OpenZeppelin. Contracts v5.0 — Upgradeable Patterns. OpenZeppelin, 2024.

Abu Dhabi Global Market, Al Maryah Island

Abu Dhabi, United Arab Emirates

https://aethelred.org

This document is provided for informational purposes in connection with the Aethelred Foundation's registration with the ADGM DLT Foundations regime. Nothing herein constitutes financial advice, an offer of securities, or a solicitation of investment. The AETHEL token is a utility token designed to facilitate computation and governance within the Aethelred protocol. All forward-looking statements are subject to risks and uncertainties.